

Heaven's Light is Our Guide

Rajshahi University of Engineering & Technology



Department of Electrical & Computer Engineering

Lab Manual

Discrete Mathematics & Numerical Methods

ECE 2214

(1.5 Credit)

Lab Manuals have been developed for the requirements of course of ECE 2214

Department of Electrical & Computer Engineering

Faculty of Electrical & Computer Engineering

Rajshahi University of Engineering & Technology

Rajshahi-6204

ECE 1102
Circuits & Systems-I Sessional
Credit: 1.50
INDEX

Sl. No.	Name of the Experiment
1	To setup Python and PyCharm in the PC and watch a tutorial and prepare a report about the key takeaways and insights after viewing the video.
2	Convert an English sentence into a logical expression using Python. Given an English sentence, you are required to convert it into a logical expression. Here is an example: "You cannot ride the roller coaster if you are under 4 feet tall unless you are older than 16 years old." $(r \wedge \neg s) \rightarrow \neg q$
3	The task of determining whether a given logical expression is a tautology or not. A tautology is a statement that is always true, regardless of the truth values of its variables.
4	Write python codes to find logical contradiction, contraposition & equivalence
5	Modular Exponentiation and Binary Arithmetic in Python.
6	Finding Chinese remainder theorem & Carmichael number using python
7,8,9	Implementation of root finding methods Bisection, False position, and Newton Raphson method in MatLab.
10	Implementation of Secant method for finding root using MatLab.
11,12	Implementation of Taylor's sine series and interpolation using MatLab.

Experiment No.: 01

Experiment name: To setup Python and PyCharm in the PC and watch a tutorial and prepare a report about the key takeaways and insights after viewing the video.

Python Basics (Concise Titles):

1. Installation & IDE: Install Python and set up PyCharm.
2. Variables: Integer, Float, String, Boolean (no type declaration needed).
3. Input: input() reads strings; convert using int() or float().
4. String Operations: find(), upper(), lower(), replace().
5. Arithmetic Operators: +, -, *, /, //, %, **.
6. Logical Operators: and, or, not.
7. Comparison Operators: >, <, >=, <=, ==, !=.
8. If Statements: Conditional blocks defined by indentation.
9. Loops: for loops over items; while loops on conditions.
10. Lists: Store mixed types; operations: append(), insert(), remove(), clear().
11. Range Function: Generate sequences with range(start, stop, step).

Output

Discussion

Conclusion

Experiment No.: 02

Experiment name: Convert an English sentence into a logical expression using Python. Given an English sentence, you are required to convert it into a logical expression. Here is an example:

"You cannot ride the roller coaster if you are under 4 feet tall unless you are older than 16 years old."

$(r \wedge \neg s) \rightarrow \neg q$

Theory: To convert an English sentence into a logical expression using python the main task is to split the sentence into smaller part and set logic for each part of the sentence. After splitting the sentence into smaller sentences, we have to set logic for the smaller sentences based on certain condition such as is the condition TRUE or FALSE. Then by comparing the logical information using if, else method we have to show the result output as a complete logical expression.

Methodology:

1. Firstly, the logic was set for smaller sentences:

Q = "YOU CAN RIDE THE ROLLER COASTER."

R = "YOU ARE UNDER 4 FEET TALL."

S = "YOU ARE OLDER THAN 16 YEARS OLD."

2. Then a function named `english_to_logic()` was created to convert the sentences into logical expression.
3. Inside the function the sentence was split into two small sentence a and b using `split()` method. Then again b sentence was split into two smaller sentence c and d using `split()` method. Sentences were split setting the splitting word if, unless.
4. Then three conditions considering if the sentences are negative or positive was set.
5. If the condition is true then we show the result as true q, r, s and when the conditions are false the we show the not symbol before logical expression for example: $\neg s$, $\neg q$ etc.
6. As unless is a negative linking word so the logical expression for the sentence after unless gets negative.

7. Then finally the result was shown as a logical expression.

Required Tools:

1. Pycharm Text Editor
2. MS office

Code Implementations:

Algorithm: English to Logic Converter

1. Start
2. Define three known statements:
 - $q = \text{"you can ride the roller coster"}$
 - $r = \text{"you are under 4 feet tall"}$
 - $s = \text{"you are older than 16 years old"}$
3. Input a sentence line from the user.
4. Convert the input line to lowercase.
5. Initialize truth values: $x = \text{True}$, $y = \text{True}$, $z = \text{True}$.
6. Split the sentence:
 - Split line into a and b using " if ".
 - Split b into c and d using " unless ".
7. Compare each part with predefined sentences:
 - If $a \neq q$, set $x = \text{False}$.
 - If $c \neq r$, set $y = \text{False}$.
 - If $d \neq s$, set $z = \text{False}$.
8. Construct logical expression:
 - Begin with "("
 - If y is False, append " $\neg r$ " else append "r".
 - Add " $\wedge$ ".
 - If z is False, append "s" else append " $\neg s$ ".

- o Close bracket and add $\rightarrow$.
 - o If x is False, append " $\neg q$ " else append "q".
9. Output the final logical expression.
 10. End

Output

Discussion

Conclusion

Experiment No.: 03

Experiment name: The task of determining whether a given logical expression is a tautology or not. A tautology is a statement that is always true, regardless of the truth values of its variables.

Theory: To convert an English sentence into a logical expression using python the main task is to split the sentence into smaller part and set logic for each part of the sentence. After splitting the sentence into smaller sentences, we have to set logic for the smaller sentences based on certain condition such as is the condition TRUE or FALSE. Then by comparing the logical information using if, else method we have to show the result output as a complete logical expression.

Methodology:

Required Tools:

3. Pycharm Text Editor
4. MS office

Code Implementations:

Algorithm: Tautology Checker

Step 1: Start

Step 2: Input

- Read a logical expression as a string.
- Read three variables: var1, var2, and var3.

Step 3: Initialize

- Set output = True
- Store the original expression in compare.

Part A: Infix to Postfix Conversion

Step 4: Define function `infix_to_postfix(string)`

1. Add parentheses around the expression.
2. Initialize an empty stack and an empty output list.
3. Split the expression into tokens.
4. For each token:
 - If token is "(", push it to the stack.
 - If token is "not", "and", or "or", push it to the stack.
 - If token is ")", pop from the stack and append to output until "(" is found.
 - Otherwise, append token to output.
5. Join output into a postfix string.
6. Call postfix_evaluation(postfix) and return the result.

Part B: Postfix Evaluation

Step 5: Define function postfix_evaluation(string)

1. Initialize an empty stack.
2. Split the expression into tokens.
3. For each token:
 - If token is "not":
 - Pop one operand from the stack.
 - Push the negated result back ($\text{True} \leftrightarrow \text{False}$).
 - If token is "and" or "or":
 - Pop two operands from the stack.
 - Convert them to Boolean values.
 - Perform the operation (and / or).
 - Push the result (as string) back to the stack.
 - Otherwise, push the token to the stack.
4. If final stack top is "True", return True, else return False.

Part C: Generate Truth Table

Step 6:

For each possible combination of truth values of var1, var2, and var3 (True/False):

1. Replace each variable in the input expression with the corresponding Boolean value.
2. Evaluate the expression using `infix_to_postfix()`.
3. Display var1, var2, var3, and the output result.
4. Update output = output AND result.

Step 7: Check Tautology

- If final output = True, print "Tautology".
- Else, print "Not Tautology".

Step 8: Stop

Output

Discussion

Conclusion

Experiment No.: 04

Experiment name: Write python codes to find logical contradiction, contraposition & equivalence

Theory: Contradiction, contraposition, and equivalence is very important part of logical operations of discrete mathematics.

Contradiction: A statement that is always false is known as a contradiction. A statement or notion that is logically or intrinsically false is referred to as a contradiction. It occurs when a logical argument leads to a situation where two opposing statements cannot both be true at the same time. One common example of a contradiction in discrete mathematics is:

"There exists an integer n such that n is both even and odd."

This statement is a contradiction because by the definition of even and odd integers, no integer can be both even and odd simultaneously. Even integers are divisible by 2, whereas odd integers are not.

Contraposition: Contraposition refers to the inference of going from a conditional statement into its logically equivalent contrapositive. The contrapositive of a statement is formed by negating both the hypothesis and the conclusion of the original conditional statement and reversing their order. Here is the general form of a contrapositive:

Original statement: If A , then B .

Contrapositive: If not B , then not A .

In other words, the contrapositive switches the roles of the antecedent (A) and the consequent (B) and negates both.

Equivalence: Equivalence in logical expressions of discrete math is a concept that deals with the relationship between two or more logical statements that have the same truth value for all possible combinations of truth values for their variables. Equivalence can be denoted by the symbol $\equiv$ and can be verified by using truth tables or logical laws.

Required Tools:

5. Pycharm Text Editor
6. MS office

Code Implementations:

1. Algorithm: Contradiction Checker

1. Start
2. Input logical expression and variables var1, var2, var3.
3. Set flag = 1.
4. For each combination of True/False values of var1, var2, var3:
 - a. Replace variables in expression with their Boolean values.
 - b. Convert infix to postfix form.
 - c. Evaluate postfix using a stack:
 - o Apply not, and, or as logical operations.
 - d. If any result is True $\rightarrow$ set flag = 0.
5. If flag = 1, print "Contradiction", else "Not Contradiction".
6. Stop

2. Algorithm: Contraposition

1. Start
2. Input conditional sentence (e.g., "A, if B").
3. Split sentence into two parts — antecedent and consequent.
4. Remove "if" and rearrange sentence.
5. Output: "If it is not true that [consequent], it is not true that [antecedent]."
6. Stop

3. Algorithm: Logical Equivalence

1. Start
2. Import sympy and define symbols p, q.

3. Define two logical expressions (e.g., $\text{expr1} = \neg(p \vee q)$, $\text{expr2} = \neg p \wedge \neg q$).
4. Simplify both using `sympy.simplify_logic()`.
5. Compare results:
 - o If equal $\rightarrow$ print "Equivalent",
 - o Else $\rightarrow$ print "Not Equivalent."
6. Stop

Output

Discussion

Conclusion

Code Implementations:

1. Algorithm: Binary Addition and Subtraction

Start

Input two binary numbers A and B.

For Addition (bin_add)

- Equalize lengths of A and B by padding zeros.
- Initialize carry = 0, result = "".
- Traverse bits from right to left:

Compute $r = \text{carry} + \text{bit}(A) + \text{bit}(B)$.

Append "1" to result if r is odd, else "0".

Update carry = 1 if $r \geq 2$, else 0.

- If carry $\neq 0$, prepend "1" to result.
- Return the final binary sum.

For Subtraction (bin_sub)

- Equalize lengths of A and B.
- Find 2's complement of B using:

Invert bits ($1 \rightarrow 0$, $0 \rightarrow 1$).

Add 1 to the result.

- Add A and complement of B using bin_add.
- If carry occurs, discard it (positive result).
- If no carry, take complement of result (negative value).
- Return final difference.

Print sum and difference of A and B.

Stop

2. Algorithm: Exponent and Modulus (Binary Exponentiation)

Start

Input base bas, exponent exp, and modulus mod.

Initialize ans = 1.

While $\text{exp} > 0$:

- a. If exp is odd $\rightarrow$ set $\text{ans} = (\text{ans} \times \text{bas}) \bmod \text{mod}$.
- b. Update $\text{bas} = (\text{bas} \times \text{bas}) \bmod \text{mod}$.
- c. Divide exp by 2 (integer division).

When loop ends, return ans .

Print the result.

Stop

Output

Discussion

Conclusion

Experiment No.: 06

Experiment name: Finding Chinese remainder theorem & Carmichael number using python.

Theory:

The Chinese remainder theorem is a mathematical result that allows one to find a unique solution to a system of congruences with coprime moduli. The Chinese remainder theorem addresses the following type of problem. One is asked to find a number that leaves a remainder of 0 when divided by 5, remainder 6 when divided by 7, and remainder 10 when divided by 12. The simplest solution is 370. Note that this solution is not unique, since any multiple of $5 \times 7 \times 12 (= 420)$ can be added to it and the result will still solve the problem.

Carmichael number is a composite number that satisfies Fermat's little theorem for every integer that is relatively prime to it. In other words, a Carmichael number is a number that behaves like a prime number in a certain way, even though it is not actually prime. For example, 561 is a Carmichael number, because it is divisible by 3, 11, and 17, but it also satisfies the equation $(a^{560} \equiv 1 \pmod{561})$ for any (a) that is not a multiple of 3, 11, or 17. Carmichael numbers are also called Fermat pseudoprimes or absolute Fermat pseudoprimes.

Required Tools:

1. MS office
2. Pycharm

Code Implementations:

1. Algorithm: Chinese Remainder Theorem
 1. Start
 2. Input arrays:
 - o $num[] \rightarrow$ list of divisors
 - o $rem[] \rightarrow$ corresponding remainders
 - o $k \rightarrow$ number of equations
 3. Initialize $x = 1$.
 4. Repeat until a valid x is found:
 - a. For each j from 0 to $k-1$:

- o If $x \% \text{num}[j] \neq \text{rem}[j]$, break loop.
 - b. If all congruences hold ($j == k$), return x as the solution.
 - c. Else, increment x by 1 and continue.
- 5. Print the smallest positive integer x satisfying all congruences.
- 6. Stop

2. Algorithm: Carmichael Number Checker

1. Start
2. Define $\text{gcd}(a, b)$:
 - o If $a < b$, swap them.
 - o If $a \% b == 0$, return b .
 - o Else, recursively call $\text{gcd}(b, a \% b)$.
3. Define $\text{power}(x, y, \text{mod})$ (modular exponentiation):
 - o If $y = 0$, return 1.
 - o Compute $\text{temp} = \text{power}(x, y // 2, \text{mod}) \% \text{mod}$.
 - o Set $\text{temp} = (\text{temp} \times \text{temp}) \% \text{mod}$.
 - o If y is odd, multiply temp by x and take modulo mod .
 - o Return temp .
4. Define $\text{isCarmichaelNumber}(n)$:
 - o For every integer b from 2 to $n-1$:
 - a. If $\text{gcd}(b, n) = 1$ (coprime), then check if $b^{(n-1)} \bmod n = 1$.
 - b. If the condition fails for any b , return 0 (not Carmichael).
 - o If true for all b , return 1 (Carmichael number).
5. Test the function with numbers (e.g., 100, 423, 1105).
6. Print whether each is a Carmichael number or not.
7. Stop

Output

Discussion

Conclusion

Experiment No.: 7, 8, 9

Experiment name: Implementation of root finding methods Bisection, False position, and Newton Raphson method in MatLab.

Theory:

Bisection Method (Concise Theory):

The bisection method is an iterative root-finding technique based on the Intermediate Value Theorem. It works for a continuous function $f(x)$ on $[a, b]$ where $f(a)$ and $f(b)$ have opposite signs, ensuring a root exists between them.

At each step, the midpoint is calculated as:

$$c = \frac{a + b}{2}$$

Then:

- If $f(a) \times f(c) < 0$, the root lies in $[a, c] \rightarrow$ set $b = c$.
- If $f(a) \times f(c) > 0$, the root lies in $[c, b] \rightarrow$ set $a = c$.

This process repeats until the interval is sufficiently small or the desired accuracy is achieved.

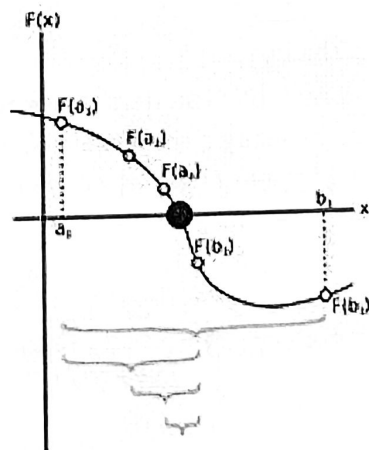


Figure 1: Graphical representation of bisection

Newton-Raphson Method (Concise Theory):

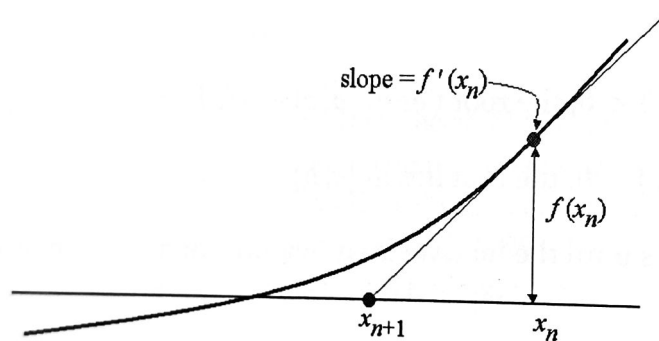
The Newton-Raphson method is an iterative technique used to find the root of a nonlinear equation $f(x) = 0$. It starts with an initial guess x_0 and refines it using the formula:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

The method is based on approximating the function by its tangent at x_n and finding where this tangent crosses the x-axis.

Conditions for use:

- $f(x)$ must be continuous and differentiable near the root.
- $f'(x) \neq 0$ at the root.
- The initial guess should be close to the actual root.



False Position Method (Concise Theory):

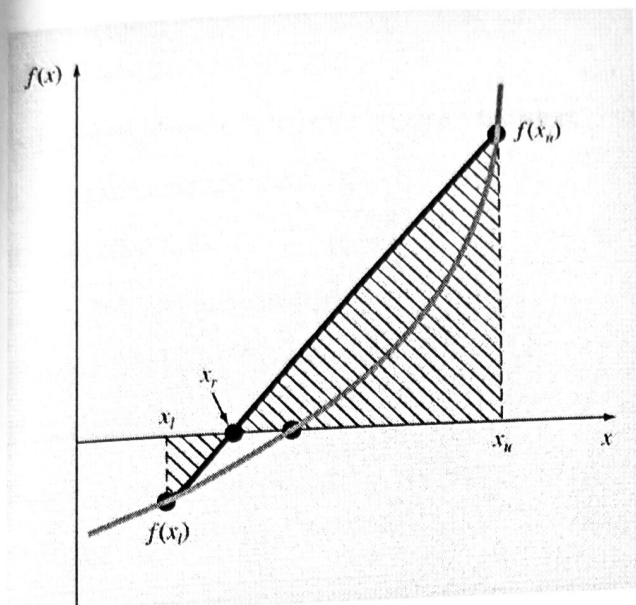
The False Position (Regula Falsi) Method is an iterative root-finding technique that improves on the bisection method by using a secant line to estimate the root. It applies to a continuous function $f(x)$ on $[a, b]$ where $f(a)$ and $f(b)$ have opposite signs.

The root is approximated as:

$$c = a - f(a) \cdot \frac{b - a}{f(b) - f(a)}$$

Then:

- If $f(a) \times f(c) < 0$, the root lies in $[a, c] \rightarrow$ set $b = c$.



- If $f(a) \times f(c) > 0$, the root lies in $[c, b] \rightarrow$ set $a = c$.

Required Tools:

3. MS office
4. MatLAB

Code Implementations:

1. Algorithm: Bisection Method

Start

Input: equation $y(x)$, initial guesses a and b .

Set tolerance $\lim = 0.001$.

Compute $f_a = y(a)$ and $f_b = y(b)$.

If $f_a * f_b > 0$, display "Wrong guess" and stop.

Else:

a. Compute midpoint $c = (a + b)/2$.

b. Evaluate $f_c = y(c)$.

c. While $|f_c| > \lim$:

Print a, b, c, fc.

If $f_a * f_c < 0$, set $b = c$, else $a = c$.

Update $c = (a + b)/2$ and $f_c = y(c)$.

Print root = c.

Stop

2. Algorithm: Secant Method

Start

Input: equation $y(x)$, initial guesses a and b.

Set tolerance $\lim = 0.001$.

Compute $f_a = y(a)$ and $f_b = y(b)$.

If $f_a * f_b > 0$, display "Wrong guess" and stop.

Else:

a. Compute $c = a - (a-b)*f_a / (f_a-f_b)$.

b. Evaluate $f_c = y(c)$.

c. While $|f_c| > \lim$:

Print a, b, c, fc.

Update either a or b depending on sign of $f_a * f_c$.

Compute next $c = a - (a-b)*f_a / (f_a-f_b)$ and $f_c = y(c)$.

Print root = c.

Stop

3. Algorithm: Newton-Raphson Method

Start

Input: equation $y(x)$, initial guess a, maximum steps N.

Set tolerance $\lim = 0.001$ and step counter = 1.

Compute derivative $g(x) = dy/dx$.

While $|y(a)| > \text{lim}$ and $\text{step} \leq N$:

- a. Evaluate $f_a = y(a)$ and $g_a = g(a)$.
- b. If $g_a = 0$, display "Division by zero" and stop.
- c. Compute next approximation: $b = a - f_a/g_a$.
- d. Print step, a, f_a .
- e. Set $a = b$ and increment step.

If $\text{step} > N$, display "Not convergent".

Print root = a.

Stop

Output

Discussion

Conclusion

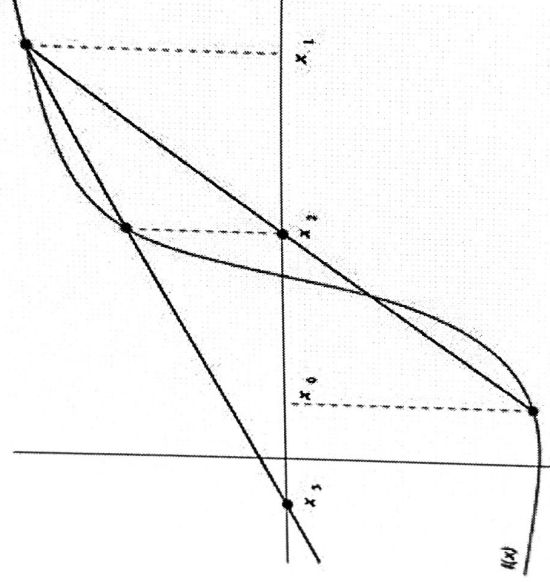
Experiment No.: 10

Experiment name: Implementation of Secant method for finding root using MatLab.

Theory:

The secant method is a way of finding an approximate solution to an equation that involves a continuous function. It works by repeatedly drawing a secant line through two points on the function and finding the x-intercept of the secant line as the next approximation of the root. The process is repeated until the desired accuracy or the maximum number of iterations is reached.

The secant method is based on the idea that a function can be approximated by a linear function near its root, and the root of the linear function is easier to find than the root of the original function. The secant method can also be derived from the Taylor series expansion of the function, or from the geometric interpretation of the secant method.



Required Tools:

5. MS office
6. MatLAB

Code Implementations:

Algorithm: Secant Method Function

Function: Root_sec_meth(func, a, b, lim)

1. Start Function
2. Evaluate $Fa = \text{func}(a)$ and $Fb = \text{func}(b)$.
3. Compute first approximation:

$$c = b - Fb \cdot \frac{(b - a)}{(Fb - Fa)}, Fc = \text{func}(c)$$

4. While $|Fc| > \text{lim}$:
 - a. Display $[a, b, c, Fc]$.
 - b. Update variables: $a = b, Fa = Fb, b = c, Fb = Fc$.
 - c. Compute next approximation:

$$c = b - Fb \cdot \frac{(b - a)}{(Fb - Fa)}, Fc = \text{func}(c)$$

5. Display last iteration $[a, b, c, Fc]$.
6. Display root = c.
7. End Function

Algorithm: Main Program

1. Start
2. Input function string func_str.
3. Convert string to function handle: $\text{func} = @(\text{x}) \dots$
4. Input first guess a, second guess b, and tolerance lim.
5. Call Root_sec_meth(func, a, b, lim).
6. Stop

Output

Discussion

Conclusion

Experiment No.: 11, 12

Experiment name: Implementation of Taylor's sine series and interpolation using MatLab.

Theory:

Taylor Series:

A function $f(x)$ can be expressed as an infinite sum of its derivatives at a point a

$$f(x) = f(a) + \frac{f'(a)}{1!}(x-a) + \frac{f''(a)}{2!}(x-a)^2 + \frac{f'''(a)}{3!}(x-a)^3 + \dots$$

- Used to approximate functions near a point a .
- Example: For $\sin(x)$ at $a = 0$ (Maclaurin series):

$$\sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots = \sum_{n=0}^{\infty} (-1)^n \frac{x^{2n+1}}{(2n+1)!}$$

Interpolation:

Interpolation estimates unknown values between known data points.

Newton Forward Interpolation:

- Uses **forward differences** of data to approximate values.
- Formula:

$$f(x) = f(x_0) + u\Delta f(x_0) + \frac{u(u-1)}{2!}\Delta^2 f(x_0) + \frac{u(u-1)(u-2)}{3!}\Delta^3 f(x_0) + \dots$$

- x_0 : first data point, $\Delta f(x_0)$: first forward difference, etc.
- $u = \frac{x-x_0}{h}$, where h is the uniform spacing between data points.

Required Tools:

1. MS office
2. MatLAB

Code Implementations:

Part 1: Taylor Series for Sine Function

1. Start
2. Input maximum order n.
3. Plot $\sin(x)$ in blue.
4. For each order 1 to n:
 - a. Initialize $\text{Taylor_s} = 0$.
 - b. For $i = 0$ to order:

$$\text{Taylor_s} = \text{Taylor_s} + \frac{(-1)^i x^{2i+1}}{(2i+1)!}$$

c. Plot Taylor_s with proper label.

5. Add title "Sine wave".
6. End Taylor series plotting

Part 2: Forward Difference Interpolation

1. Start
2. Input number of data pairs.
3. Create matrix of x and y values.
4. Compute step size $h = x_2 - x_1$.
5. Input x to evaluate function.
6. Compute $p = (x - x_0)/h$, where $x_0 = \text{first } x$.
7. Construct forward difference table:

o For each order, compute differences recursively and store in matrix.

8. Initialize $y_0 = \text{first } y \text{ value}$.

9. Compute interpolated value using:

$$y_0 = y_0 + \frac{p \cdot \Delta y}{1!} + \frac{p(p-1) \cdot \Delta^2 y}{2!} + \dots$$

10. Display difference table and interpolated value y_0 .

11. Stop

Output

Discussion

Conclusion